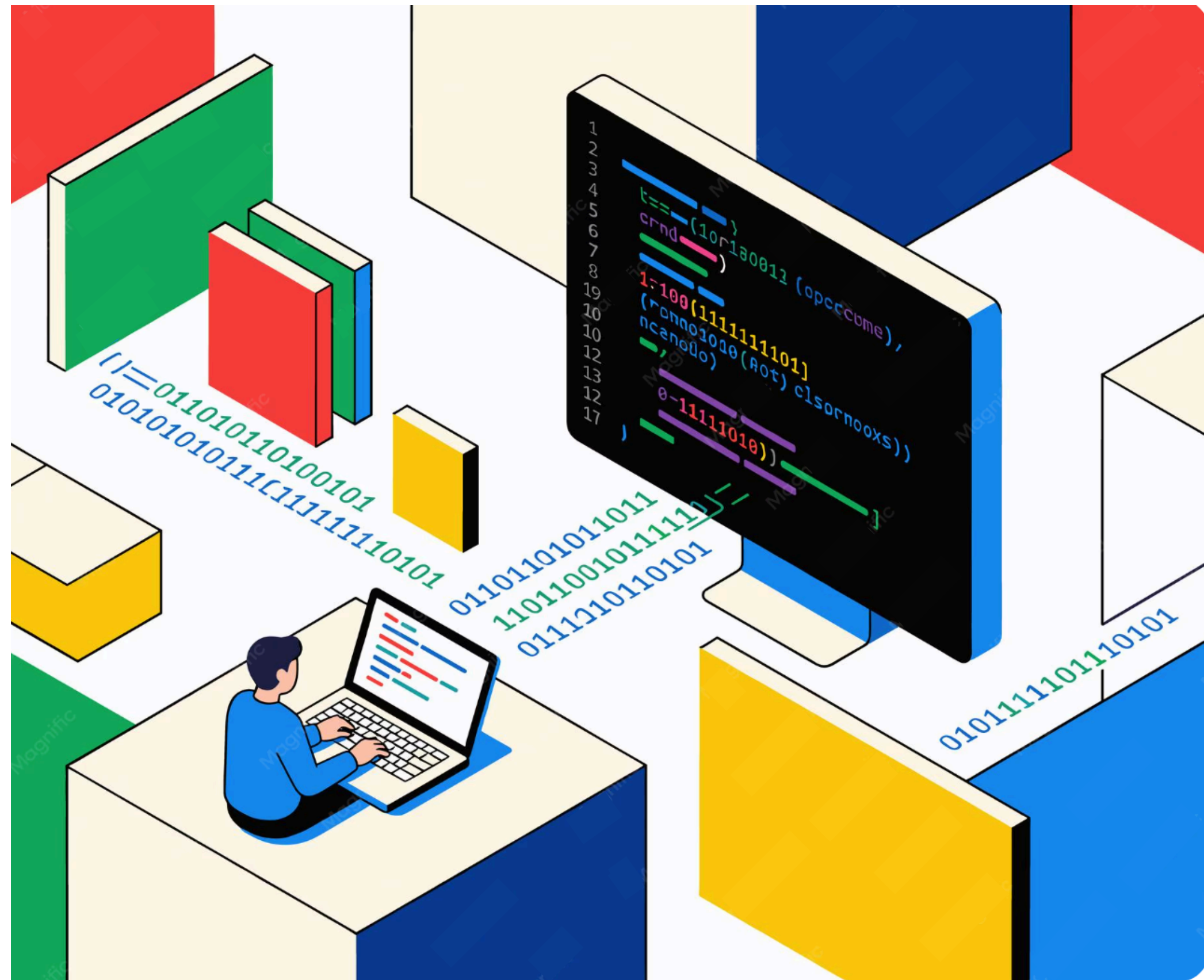


Pemrograman Berbasis Objek

Class and Object

Semester 2026/2027

31 Agustus 2026



Reference

- 1 Lutz, M. (2025). Learning Python: Powerful object-oriented programming (6th ed.). O'Reilly Media.
- 2 Phillips, D. (2018). Python 3 object-oriented programming (3rd ed.). Packt Publishing.
- 3 McLaughlin, B., Pollice, G., & West, D. (2007). Head first object-oriented analysis and design. O'Reilly Media.

Studi Kasus

- Anda diminta membuat program untuk menyimpan data mahasiswa.
- Empat hal ini perlu disimpan untuk setiap mahasiswa:
 - Nama,
 - NIM,
 - Angkatan, dan
 - Daftar mata kuliah yang diambil.

Bagaimana menulis program tersebut dalam
bahasa python?



Kartu Data Mahasiswa

NAMA

Wahyu Austin

NIM

25123456789

ANGKATAN

2025

MATA KULIAH

1. Matematika Dasar
2. Matematika Diskrit
3. Aljabar Matriks
4. Pemrograman Dasar
5. Interaksi Manusia dan Kecerdasan Artifisial
6. Literasi Digital
7. Pendidikan Jasmani dan Kebugaran

Studi Kasus

- Program sederhana :

```
nama = []
nim = []
angkatan = []
matkul = []

def tambah_mahasiswa(nama_baru, nim_baru, angkatan_baru, matkul_baru):
    nama.append(nama_baru)
    nim.append(nim_baru)
    angkatan.append(angkatan_baru)
    matkul.append(matkul_baru)
```

Studi Kasus

- Program sederhana :

```
tambah_mahasiswa(  
    "Andi",  
    "25123456789",  
    2023,  
    ["Matematika Diskrit", "Dasar Pemrograman"]  
)
```

```
tambah_mahasiswa(  
    "Budi",  
    "25987654321",  
    2023,  
    ["Basis Data", "00P"]  
)
```

Studi Kasus

- Tetapi perhatikan struktur datanya:

```
nama = ["Andi", "Budi", "Citra"]
nim = ["25123456789", "25987654321", "25111222333"]
angkatan = [2025, 2025, 2025]
matkul = [
    ["Matematika Diskrit", "Dasar Pemrograman"],
    ["Basis Data", "OOP"],
    ["Algoritma"]
]
```

Bagaimana hubungan antar-data?

→ **Hanya disimpan berdasarkan posisi/index.**

Studi Kasus

- Solusi lain bisa dijadikan mengusulkan menjadi bentuk dictionary:

```
mahasiswa = {  
    "nama": "Andi",  
    "nim": "25012345678",  
    "angkatan": 2025,  
    "matkul": [  
        "Matematika Diskrit",  
        "Dasar Pemrograman"  
    ]  
}
```

Studi Kasus

- Solusi lain bisa dijadikan mengusulkan menjadi bentuk dictionary:

```
mahasiswa = {  
    "nama": "Andi",  
    "nim": "25012345678",  
    "angkatan": 2025,  
    "matkul": [  
        "Matematika Diskrit",  
        "Dasar Pemrograman"  
    ]  
}
```

Lalu bagaimana jika data mahasiswa yang bukan hanya menyimpan informasi, tetapi juga membutuhkan berbagai operasi:

- Tambah mata kuliah,
- Hapus mata kuliah,
- Cek KRS,
- Tampilkan profil,
- Hitung jumlah SKS,
- dan sebagainya.

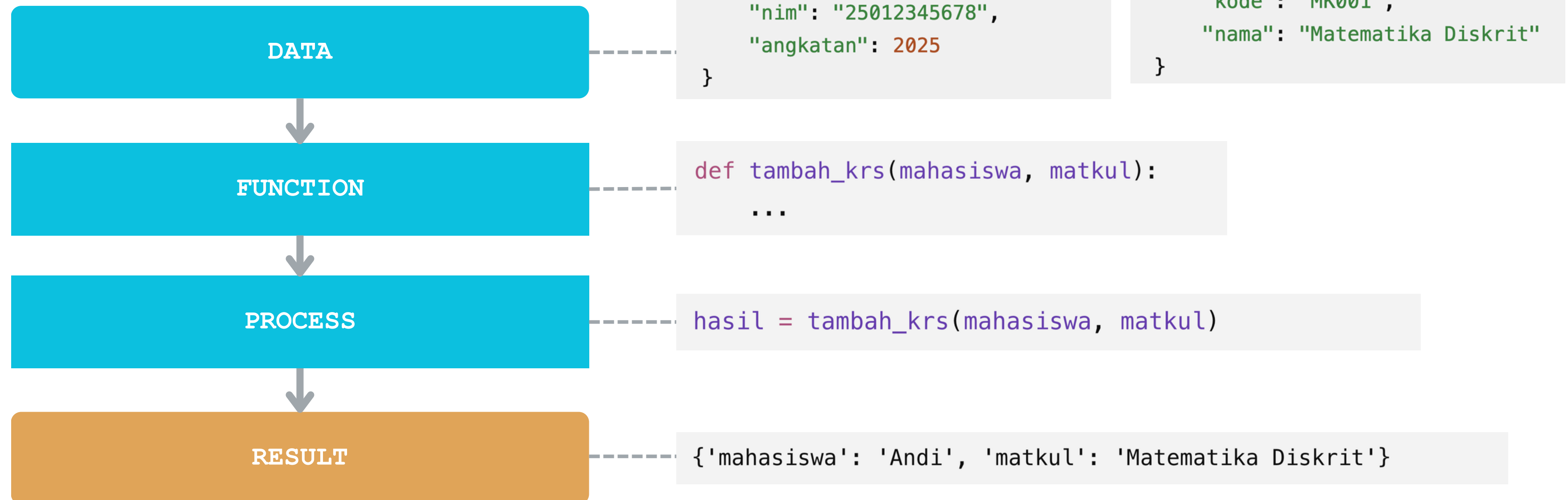
Bagaimana kalau **data** dan **fungsi** yang berkaitan dengan data tersebut **disatukan**?

Studi Kasus

- Tinggal buat saja function untuk masing2 kebutuhan :

```
def tambah_matkul(mahasiswa, matkul):  
    mahasiswa["matkul"].append(matkul)  
  
def hapus_matkul(mahasiswa, matkul):  
    mahasiswa["matkul"].remove(matkul)  
  
def tampilkan_profil(mahasiswa):  
    print("Nama:", mahasiswa["nama"])  
    print("NIM:", mahasiswa["nim"])  
    print("Angkatan:", mahasiswa["angkatan"])  
  
def cek_krs(mahasiswa):  
    print("Mata kuliah:", mahasiswa["matkul"])  
  
def hitung_sks(mahasiswa):  
    return len(mahasiswa["matkul"]) * 3
```

Pemrograman Prosedural



Pemrograman Prosedural **fokus pada proses**

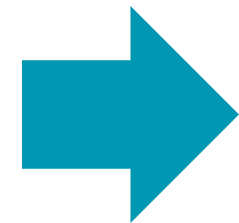
Studi Kasus

- Kemudian bagaimana jika data yang dihimpun lebih kompleks
 - 20.000 mahasiswa
 - 500 mata kuliah
 - 1000 dosen
 - Data kelas
 - KRS
 - Jadwal Perkuliahan
 - Nilai
 - Program studi
 - Fakultas

Apakah bisa diselesaikan dengan
Pemrograman Prosedural?

Studi Kasus

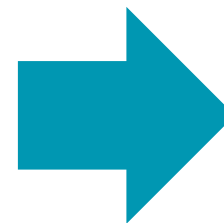
- Kemudian bagaimana jika data yang dihimpun lebih kompleks
 - 20.000 mahasiswa
 - 500 mata kuliah
 - 1000 dosen
 - Data kelas
 - KRS
 - Jadwal Perkuliahan
 - Nilai
 - Program studi
 - Fakultas



Pemrograman Berorientas Objek

Studi Kasus

- Kemudian bagaimana jika data yang dihimpun lebih kompleks
 - 20.000 mahasiswa
 - 500 mata kuliah
 - 1000 dosen
 - Data kelas
 - KRS
 - Jadwal Perkuliahan
 - Nilai
 - Program studi
 - Fakultas



- **OBJEK APA SAJA** yang ada dalam masalah kita?"

Mahasiswa

Dosen

Kelas

Mata Kuliah

Program Studi

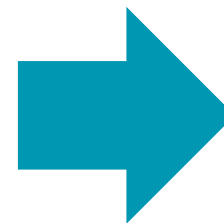
Fakultas

Lalu tentukan :

- Data apa yang dimiliki objek?
- Apa yang dapat dilakukan objek?
- Bagaimana objek berinteraksi dengan objek lain?

Studi Kasus

- Kemudian bagaimana jika data yang dihimpun lebih kompleks
 - 20.000 mahasiswa
 - 500 mata kuliah
 - 1000 dosen
 - Data kelas
 - KRS
 - Jadwal Perkuliahan
 - Nilai
 - Program studi
 - Fakultas



- **OBJEK APA SAJA** yang ada dalam masalah kita?"

Mahasiswa

Dosen

Kelas

Mata Kuliah

Program Studi

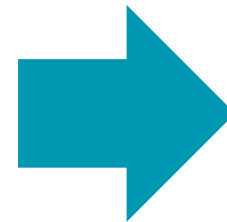
Fakultas

Lalu tentukan :

- Data apa yang dimiliki objek?
- Apa yang dapat dilakukan objek?
- Bagaimana objek berinteraksi dengan objek lain?

Studi Kasus

- Kemudian bagaimana jika data yang dihimpun lebih kompleks
 - 20.000 mahasiswa
 - 500 mata kuliah
 - 1000 dosen
 - Data kelas
 - KRS
 - Jadwal Perkuliahan
 - Nilai
 - Program studi
 - Fakultas



- Kemudian buat relasi sebagai berikut :

Mahasiswa

|— mengambil → MataKuliah
|— memiliki → KRS
|— mendapatkan → Nilai

Dosen

|— mengajar → MataKuliah

Mata Kuliah

|— memiliki → Kelas

Memodelkan mahasiswa sebagai objek

Mahasiswa

ATTRIBUTE

- nama
- NIM
- angkatan
- mata kuliah

BEHAVIOR / METHOD

- tambah_matkul()
- hapus_matkul()
- cek_krs()
- hitung_sks()

Object = data / state + behavior

A new programming
paradigm :

Object Oriented Programming

- Pendekatan pemrograman untuk **mengatasi kompleksitas program yang semakin meningkat**.
- OOP memandang program sebagai **sekumpulan objek yang saling berinteraksi** untuk mencapai tujuan tertentu.
- Setiap objek memiliki **atribut (data)** dan **method (perilaku)**.
- Python merupakan bahasa pemrograman yang berorientasi objek.

Object Oriented Programming (OOP)

- How to make OOP in Python?

- **Class** adalah cetak biru (template) untuk membuat objek.
- **Object** yang dibuat dari suatu class disebut instance.
- Class menentukan dua hal utama:
 - **Attribute** → data/properti yang dimiliki objek.
 - **Method** → operasi/perilaku yang dapat dilakukan objek.

```
class Mahasiswa:
    def __init__(self, nama, nim, angkatan):
        self.nama = nama
        self.nim = nim
        self.angkatan = angkatan
        self.matkul = []

    def tambah_matkul(self, matkul):
        self.matkul.append(matkul)

    def hitung_sks(self):
        return len(self.matkul) * 3
```

Object Oriented Programming (OOP)

1. Inisialisasi Class

- Semua Class dimulai dengan menuliskan kata kunci `class`, yang diikuti dengan nama kelas dan titik dua (:).
- Setiap baris kode yang menjorok ke kanan di bawah baris class dianggap sebagai bagian dari badan class.

```
class Mahasiswa:  
    pass
```

Object Oriented Programming (OOP)

2. Mendefinisikan Atribut

- **`__init__()`** digunakan untuk **menginisialisasi atribut objek**.
- Parameter pertama pada `__init__()` selalu berupa **`self`**, yang merujuk pada **objek yang sedang dibuat**.
- Saat objek dibuat, `self` secara otomatis merujuk pada instance tersebut sehingga atribut dapat didefinisikan pada objek.

```
class Mahasiswa:
    def __init__(self, nama, nim, angkatan):
        self.nama = nama
        self.nim = nim
        self.angkatan = angkatan
        self.matkul = []
```

Object Oriented Programming (OOP)

2. Mendefinisikan Atribut

- **Attribute Class** adalah atribut yang memiliki **nilai yang sama untuk semua instance**.
- Didefinisikan di dalam class, tetapi di luar `__init__()`.

```
class Mahasiswa:  
    universitas = "Universitas Negeri Surabaya"  
    def __init__(self, nama, nim, angkatan):  
        self.nama = nama  
        self.nim = nim  
        self.angkatan = angkatan  
        self.matkul = []
```

Object Oriented Programming (OOP)

3. Membuat Object

- **Instansiasi** adalah proses **membuat objek (instance)** dari sebuah class.
- Objek dibuat dengan menuliskan **nama class diikuti ()**.

```
# Membuat Class
```

```
class Mahasiswa:
```

```
    universitas = "Universitas Negeri Surabaya"
```

```
    def __init__(self, nama, nim, angkatan):
```

```
        self.nama = nama
```

```
        self.nim = nim
```

```
        self.angkatan = angkatan
```

```
        self.matkul = []
```

```
# Membuat Object
```

```
mhs1 = Mahasiswa("Andi", "25123456788", 2025)
```

```
mhs2 = Mahasiswa("Budi", "25123456789", 2025)
```

Object Oriented Programming (OOP)

3. Membuat Object

- **Instansiasi** adalah proses **membuat objek (instance)** dari sebuah class.
- Objek dibuat dengan menuliskan **nama class diikuti ()**.

```
# Membuat Class
class Mahasiswa:
    universitas = "Universitas Negeri Surabaya"
    def __init__(self, nama, nim, angkatan):
        self.nama = nama
        self.nim = nim
        self.angkatan = angkatan
        self.matkul = []

# Membuat Object
mhs3 = Mahasiswa()
```

Traceback (most recent call last):

File "<pyshell#6>", line 1, in <module>

Cat()

TypeError: __init__() missing 3 required positional arguments: 'nama', 'nim', and 'angkatan'

Object Oriented Programming (OOP)

4. Akses Atribut

- Saat objek Mahasiswa dibuat, Python membuat instance baru dan meneruskannya sebagai self ke **`__init__()`**.
- **Atribut instance** dapat diakses menggunakan **notasi titik (.)**.

```
# Membuat Class
class Mahasiswa:
    universitas = "Universitas Negeri Surabaya"
    def __init__(self, nama, nim, angkatan):
        self.nama = nama
        self.nim = nim
        self.angkatan = angkatan
        self.matkul = []

# Membuat Object
mhs1 = Mahasiswa("Andi", "25123456788", 2025)

# Akses Atribut
print(mhs1.nama)
print(mhs1.nim)
print(mhs1.angkatan)
```

Object Oriented Programming (OOP)

4. Mengubah Atribut

- Ubah atribut dengan memanggil atribut kelas dan memberikan nilai baru pada atribut tersebut.

```
# Membuat Class
class Mahasiswa:
    universitas = "Universitas Negeri Surabaya"
    def __init__(self, nama, nim, angkatan):
        self.nama = nama
        self.nim = nim
        self.angkatan = angkatan
        self.matkul = []

# Membuat Object
mhs1 = Mahasiswa("Andi", "25123456788", 2025)

# Mengubah Atribut
mhs1.angkatan = 2024
print(mhs1.angkatan)
```

Object Oriented Programming (OOP)

5. Mendefinisikan Instance Method

- **Instance method** adalah **fungsi** yang didefinisikan di dalam class dan digunakan oleh objek (instance).
- Parameter pertama instance method selalu **self**, yang merujuk pada objek yang memanggil method tersebut.
- Instance method digunakan untuk mendefinisikan **perilaku/operasi yang dapat dilakukan objek**.

```
# Membuat Class
class Mahasiswa:
    universitas = "Universitas Negeri Surabaya"
    def __init__(self, nama, nim, angkatan):
        self.nama = nama
        self.nim = nim
        self.angkatan = angkatan
        self.matkul = []
    def tambah_matkul(self, matkul):
        self.matkul.append(matkul)

    def hitung_sks(self):
        return len(self.matkul) * 3

# Membuat Object
mhs1 = Mahasiswa("Andi", "25123456788", 2025)

# Memanggil Instance Method
mhs1.tambah_matkul("Matematika Diskrit")
print(mhs1.matkul)
```

Penggunaan OOP pada Pemrograman Kecerdasan Artifisial


[Install](#) [User Guide](#) [API](#) [Examples](#) [Community](#) [More](#)




1.9.0 (stable)

scikit-learn

Machine Learning in Python

[Getting Started](#)
[Release Highlights for 1.9](#)

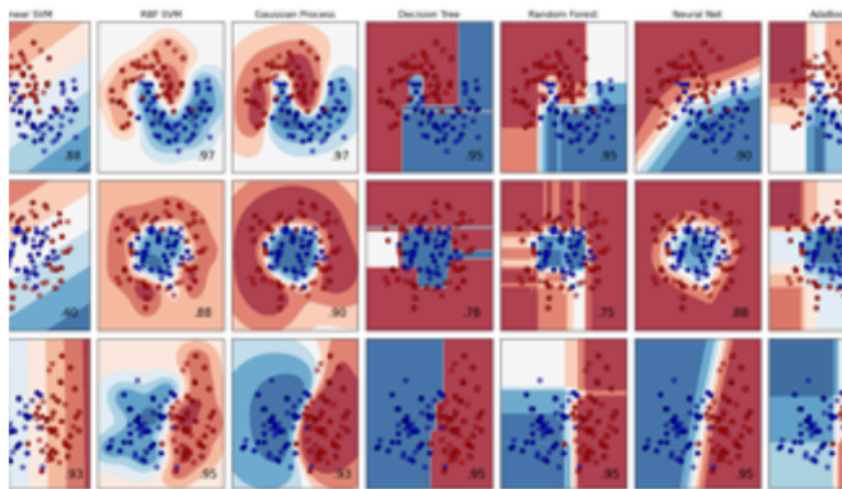
- Simple and efficient tools for predictive data analysis
- Accessible to everybody, and reusable in various contexts
- Built on NumPy, SciPy, and matplotlib
- Open source, commercially usable - BSD license

Classification

Identifying which category an object belongs to.

Applications: Spam detection, image recognition.

Algorithms: [Gradient boosting](#), [nearest neighbors](#), [random forest](#), [logistic regression](#), and [more...](#)



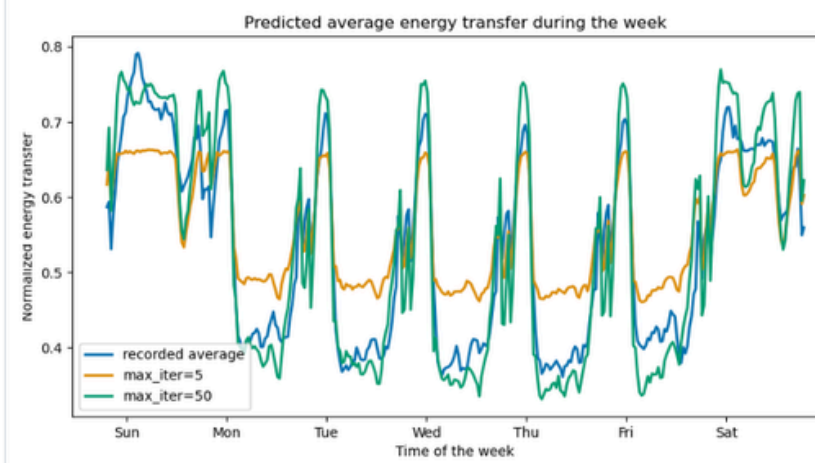
Examples

Regression

Predicting a continuous-valued attribute associated with an object.

Applications: Drug response, stock prices.

Algorithms: [Gradient boosting](#), [nearest neighbors](#), [random forest](#), [ridge](#), and [more...](#)



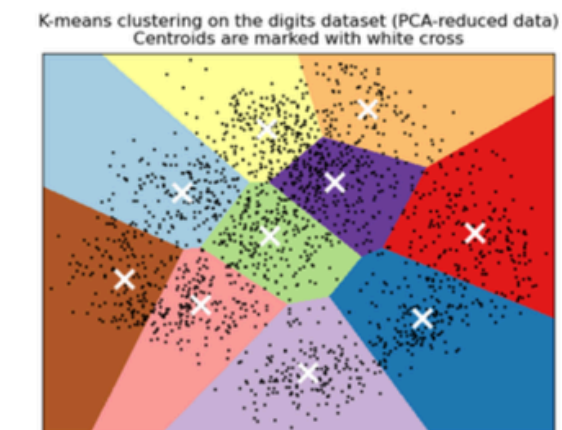
Examples

Clustering

Automatic grouping of similar objects into sets.

Applications: Customer segmentation, grouping experiment outcomes.

Algorithms: [k-Means](#), [HDBSCAN](#), [hierarchical clustering](#), and [more...](#)



Examples

Penggunaan OOP pada Pemrograman Kecerdasan Artifisial

Model Machine Learning

```
|—— parameter
|—— weights
|—— behavior
    |—— train()
    |—— predict()
    |—— evaluate()
```

- Neural Networks using sklearn

```
model = NeuralNetwork(...)
model.train(X_train, y_train)
model.predict(X_test)
model.evaluate(X_test, y_test)
```

```
class NeuralNetwork:
    def __init__(self, input_size, hidden_size, output_size):
        self.input_size = input_size
        self.hidden_size = hidden_size
        self.output_size = output_size
        self.learning_rate = 0.01

    def train(self, X, y):
        ...

    def predict(self, X):
        ...

    def evaluate(self, X, y):
        ...

model1 = NeuralNetwork(4, 8, 3)
model1.train(X_train, y_train)
prediction = model1.predict(X_test)
model1.evaluate(X_test, y_test)
```

Penggunaan OOP pada Pemrograman Kecerdasan Artifisial

Navigation

C++

Python API

torch

Aliases in
torch.nn

torch.nn

Buffer

Parameter

UninitializedPara
meter

UninitializedBuffe
r

Module

Sequential

ModuleList

ModuleDict

ParameterList

ParameterDict

Convolution Layers

`nn.Conv1d`

Applies a 1D convolution over an input signal composed of several input planes.

`nn.Conv2d`

Applies a 2D convolution over an input signal composed of several input planes.

`nn.Conv3d`

Applies a 3D convolution over an input signal composed of several input planes.

`nn.ConvTranspose1d`

Applies a 1D transposed convolution operator over an input image composed of several input planes.

`nn.ConvTranspose2d`

Applies a 2D transposed convolution operator over an input image composed of several input planes.

`nn.ConvTranspose3d`

Applies a 3D transposed convolution operator over an input image composed of several input planes.

`nn.LazyConv1d`

A `torch.nn.Conv1d` module with lazy initialization of the `in_channels` argument.

`nn.LazyConv2d`

A `torch.nn.Conv2d` module with lazy initialization of the `in_channels` argument.

`nn.LazyConv3d`

A `torch.nn.Conv3d` module with lazy initialization of the `in_channels` argument.

`nn.LazyConvTranspose1d`

A `torch.nn.ConvTranspose1d` module with lazy initialization of the `in_channels` argument.

`nn.LazyConvTranspose2d`

A `torch.nn.ConvTranspose2d` module with lazy initialization of the `in_channels` argument.